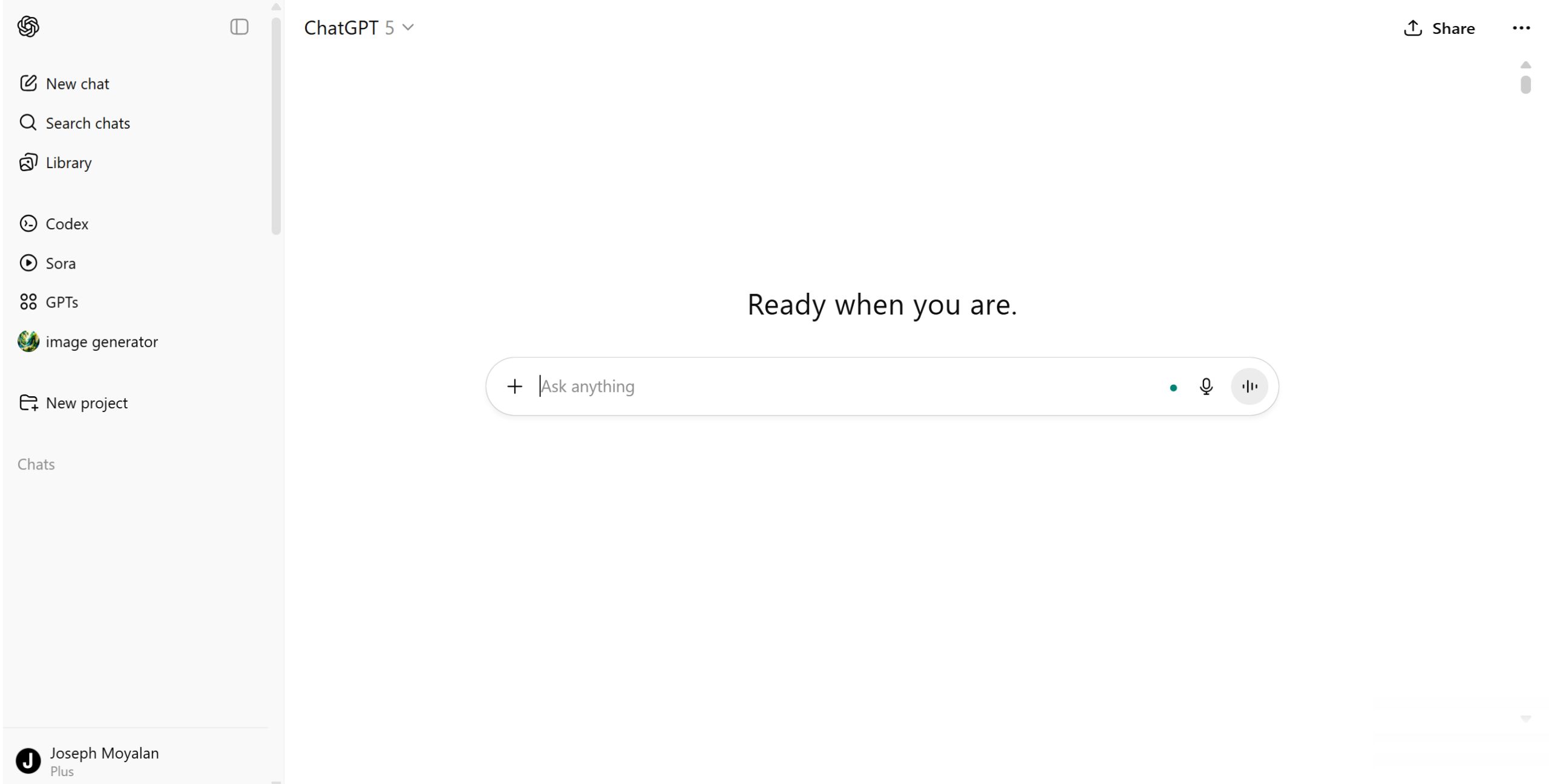
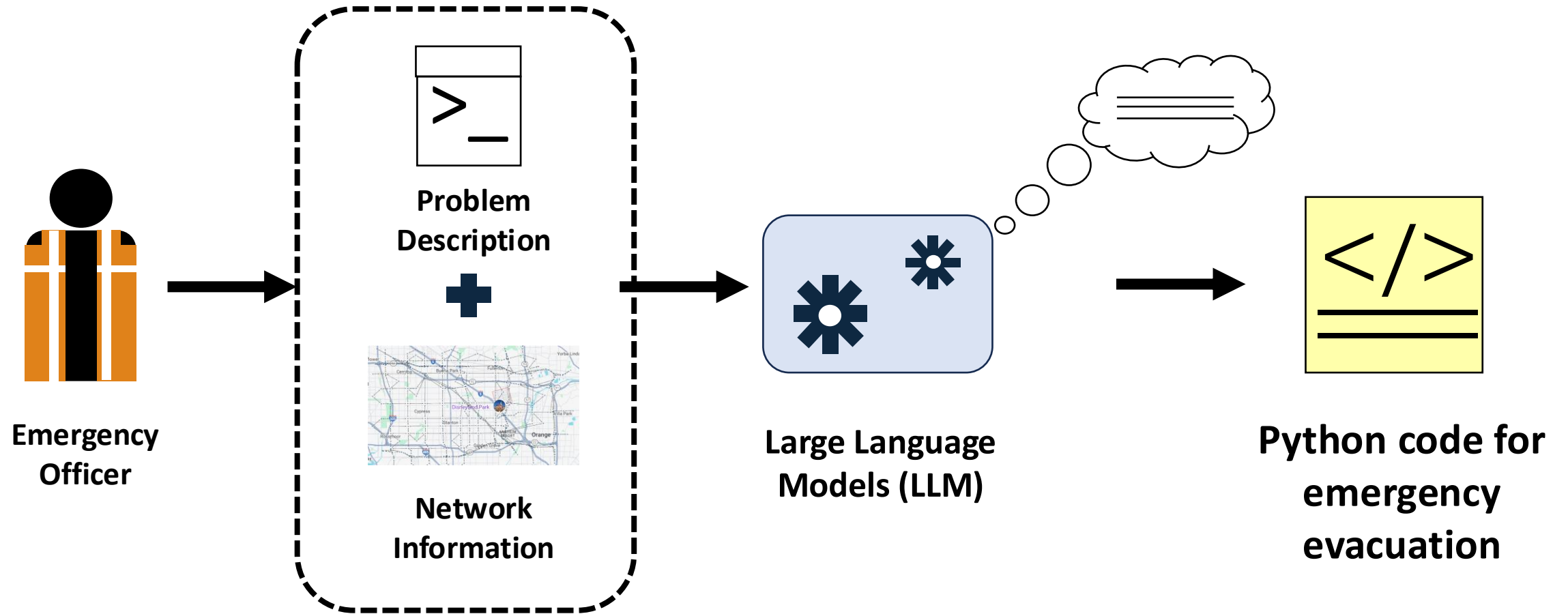


How to use LLMs for preliminary evacuation of communities? Opportunities and pitfalls

presented by

Joseph Moyalán
(University of California, Merced)





ChatGPT 5

Share

New chat

Search chats

Library

Codex

Sora

GPTs

image generator

New project

Chats

Joseph Moyalán
Plus

Provided the Anaheim data file

anaheim.geojson

File

I have this geojson file

Got it — I see you uploaded a GeoJSON file called `anaheim.geojson`.

What would you like me to do with it? For example, I can:

- Summarize the contents (features, geometry types, bounding box).
- Extract attribute fields (like names, IDs, categories).
- Visualize it on a map.
- Convert it to another format (CSV, shapefile, etc.).
- Run spatial operations (filtering, bounding box, clustering, distance calculations).

What's your goal with this file?

ChatGPT 5

Share

New chat

Search chats

Library

Codex

Sora

GPTs

image generator

New project

Chats

Joseph Moyalan
Plus

Asked to plot the network

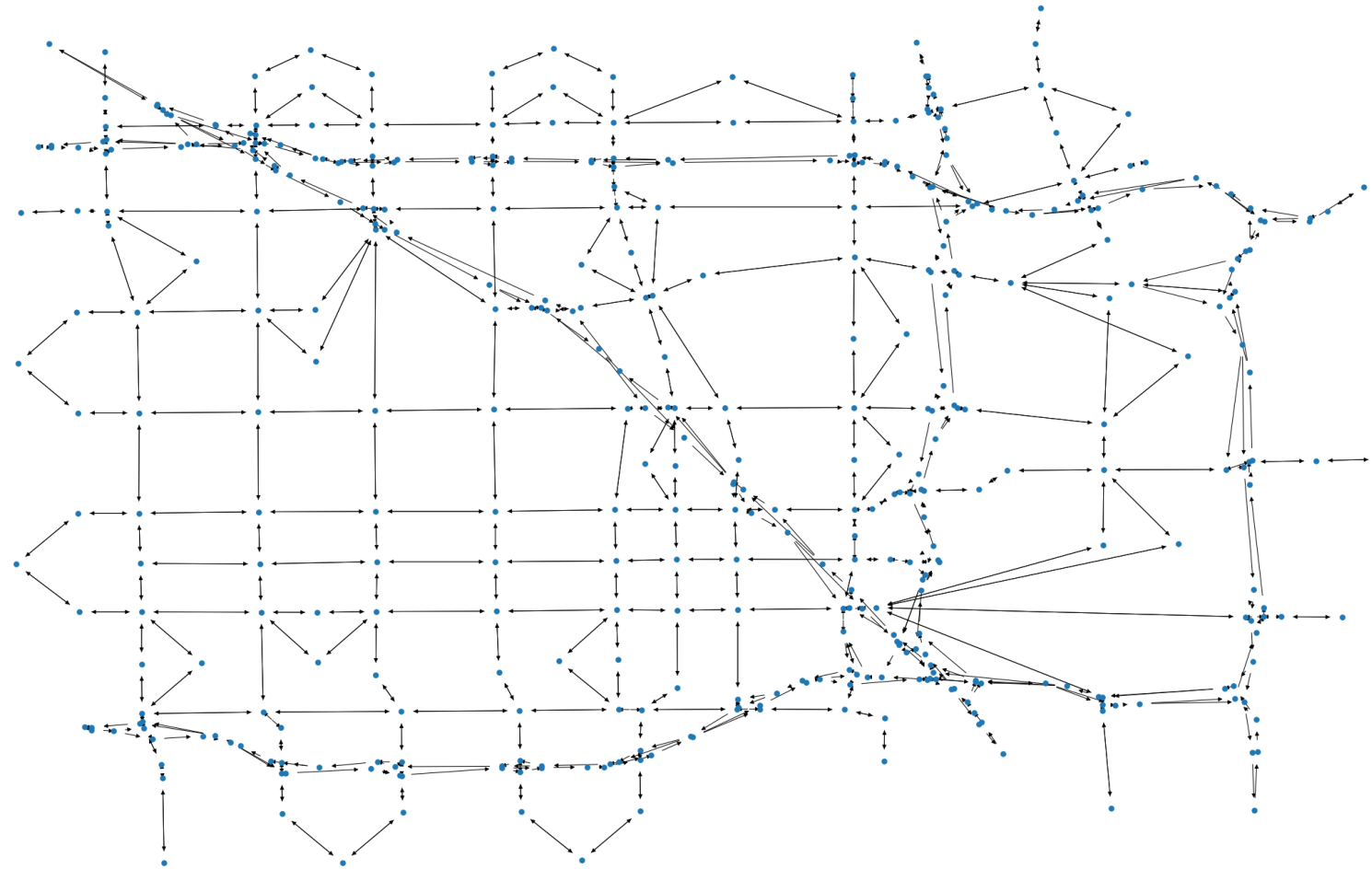
I want to build a graph network using the Excel sheet, where each edge is unidirectional and starts at the "init_node" and ends at the "term_node". Can I do it using the networkx Python package? The position coordinates for "init_node" are given in column "x1" for longitude and "y1" for latitude. Similarly, the position coordinates for "term_node" are shown in column "x2" for longitude and "y2" for latitude. Could you provide me with python code that can do this.

Yes—you can do this cleanly with NetworkX. I made you a ready-to-run script that:

- Reads the Excel you generated (`anaheim_edges.xlsx`)
- Builds a **directed** graph (each row is a one-way edge: `init_node → term_node`)
- Stores node positions (`x =lon`, `y =lat`, plus `pos=(lon,lat)`)
- Attaches edge attributes (`capacity` , `length` , `free_flow_time` , and a simple two-point line in `geometry_coords`)
- Optionally plots the graph
- Saves a GraphML file for reuse

Download the script:

[build_graph_from_excel.py](#)



Anaheim network

New chat

Search chats

Library

Codex

Sora

GPTs

image generator

New project

Chats

Joseph Moyalan Plus

ChatGPT 5

Add charging station information to the network

Add edge information to the network

I am uploading an Excel file called "FCS_location" containing the location of the fixed charging station (FCS) for the above graph. Can you put the fixed charging stations in the middle of the edges of the above graph as yellow nodes? The edge where the FCS will go has the starting node given in the column named "Node 1" and the ending node given in the column named "Node 2" or vice-versa. Could you modify the above code to include this in the graph using networkx?

Could you also modify the code so that each edge of the graph has the following features: capacity, length, and free_flow_time. These features will be used later for path planning. These features are present in the "anaheim_edges" Excel file under the column headers "capacity", "length", and "free_flow_time". Make sure to divide the "length" and "free_flow_time" by 2 for the subsequent edges whenever an FCS node is placed at the middle of an edge.

Thought for 24s

Absolutely. I've added support for your `FCS_location.xlsx` so the fixed charging stations get placed at the **midpoints** of the specified edges (yellow nodes). The code accepts `(Node 1, Node 2)` in either order and will find the corresponding edge regardless of direction.

Download the updated script:
[build_graph_with_fcs.py](#)

Share

```
File Edit Selection View Go Run Terminal Help
anaheim.geojson anaheim_nodes.geojson build_graph_from_excel.py build_graph_with_fcs.py X

C: > Users > josep > OneDrive > Desktop > AI for power systems and emergency evacuation > chatgpt test files > build_graph_with_fcs.py > build_graph_from_excel

31 def build_graph_from_excel(
77
78 capacity = float(row[capacity_col]) if pd.notna(row[capacity_col]) else None capacity_col = 'capacity'
79 length = float(row[length_col]) if pd.notna(row[length_col]) else None length_col = 'length'
80 fft = float(row[fft_col]) if pd.notna(row[fft_col]) else None row = init_node 1.000000 x1 -117.880142 y1
81
82 attrs = {"capacity": capacity, "length": length, "free_flow_time": fft} capacity = 9000.0, length = 5280.0, fft = 1.090458488
83
84 x1 = row.get(x1_col, None); y1 = row.get(y1_col, None) x1_col = 'x1', y1_col = 'y1'
85 x2 = row.get(x2_col, None); y2 = row.get(y2_col, None) x2_col = 'x2', y2_col = 'y2'
86 if pd.notna(x1) and pd.notna(y1) and pd.notna(x2) and pd.notna(y2):
87     attrs["geometry_coords"] = [(float(x1), float(y1)), (float(x2), float(y2))] x1 = np.float64(-117.8801417137077), y1 = np.float64(33.871155530597)
88
89 # G.add_edge(u, v, **attrs)
90 G.add_edge(u, v, *attrs) u = '1', v = '117', attrs = {'capacity': 9000.0, 'length': 5280.0, 'free_flow_time': 1.090458488, 'geometry_coords': [(...)]

Exception has occurred: TypeError X
DiGraph.add_edge() takes 3 positional arguments but 7 were given
File "C:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files\build_graph_with_fcs.py", line 90, in
build_graph_from_excel
G.add_edge(u, v, *attrs)
File "C:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files\build_graph_with_fcs.py", line 218, in <module>
G = build_graph_from_excel(edges_excel)
TypeError: DiGraph.add_edge() takes 3 positional arguments but 7 were given

91
92 return G
93

[0] Placed FCS_16_263_1 at midpoint of 16->263 & 263->16; pos=(-117.85248136854375, 33.76711084966432)
PS C:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files> c:; cd 'c:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files'; & 'c:\Users\josep\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files\build_graph_with_fcs.py'
Backend tkagg is interactive backend. Turning interactive mode on.
PS C:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files> c:; cd 'c:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files'; & 'c:\Users\josep\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\josep\OneDrive\Desktop\AI for power systems and emergency evacuation\chatgpt test files\build_graph_with_fcs.py'
Backend tkagg is interactive backend. Turning interactive mode on.
```



ChatGPT 5 ▾

Share



New chat



Search chats



Library



Codex



Sora



GPTs



image generator



New project

Chats

Joseph Moyalán
Plus

TypeError: DiGraph.add_edge() takes 3 positional arguments but 7 were given

That error means you're passing too many positional arguments into `DiGraph.add_edge()`.

The code was written to accept either `(u,v)` or `(v,u)` and split edges in both directions if they exist. But if your Excel has **numeric node IDs stored as integers** while the graph has them as **strings**, the match might fail.

✅ Fix: Ensure both sides are converted consistently to strings:

python

Copy code

```
print("Graph has", G.number_of_nodes(), "nodes,", G.number_of_edges(), "edges")
print("FCS nodes found:", sum(1 for _, d in G.nodes(data=True) if d.get("type") == "FCS"))

# Check one edge that was supposed to get an FCS
for _, row in pd.read_excel("FCS_location.xlsx").iterrows():
    print("Checking edge:", row["Node 1"], "-", row["Node 2"])
    if str(row["Node 1"]) in G and str(row["Node 2"]) in G:
        print("Edge exists? u->v:", G.has_edge(str(row["Node 1"]), str(row["Node 2"])),
              "v->u:", G.has_edge(str(row["Node 2"]), str(row["Node 1"])))
```

New chat

Search chats

Library

Codex

Sora

GPTs

image generator

New project

Chats

Joseph Moyalan
Plus

ChatGPT 5

I see that FCS_nodes is empty. But I do not why. For example, the first node pair in the FCS_Location file which is (16,263) can be found in the 19th row of anaheim_edges file

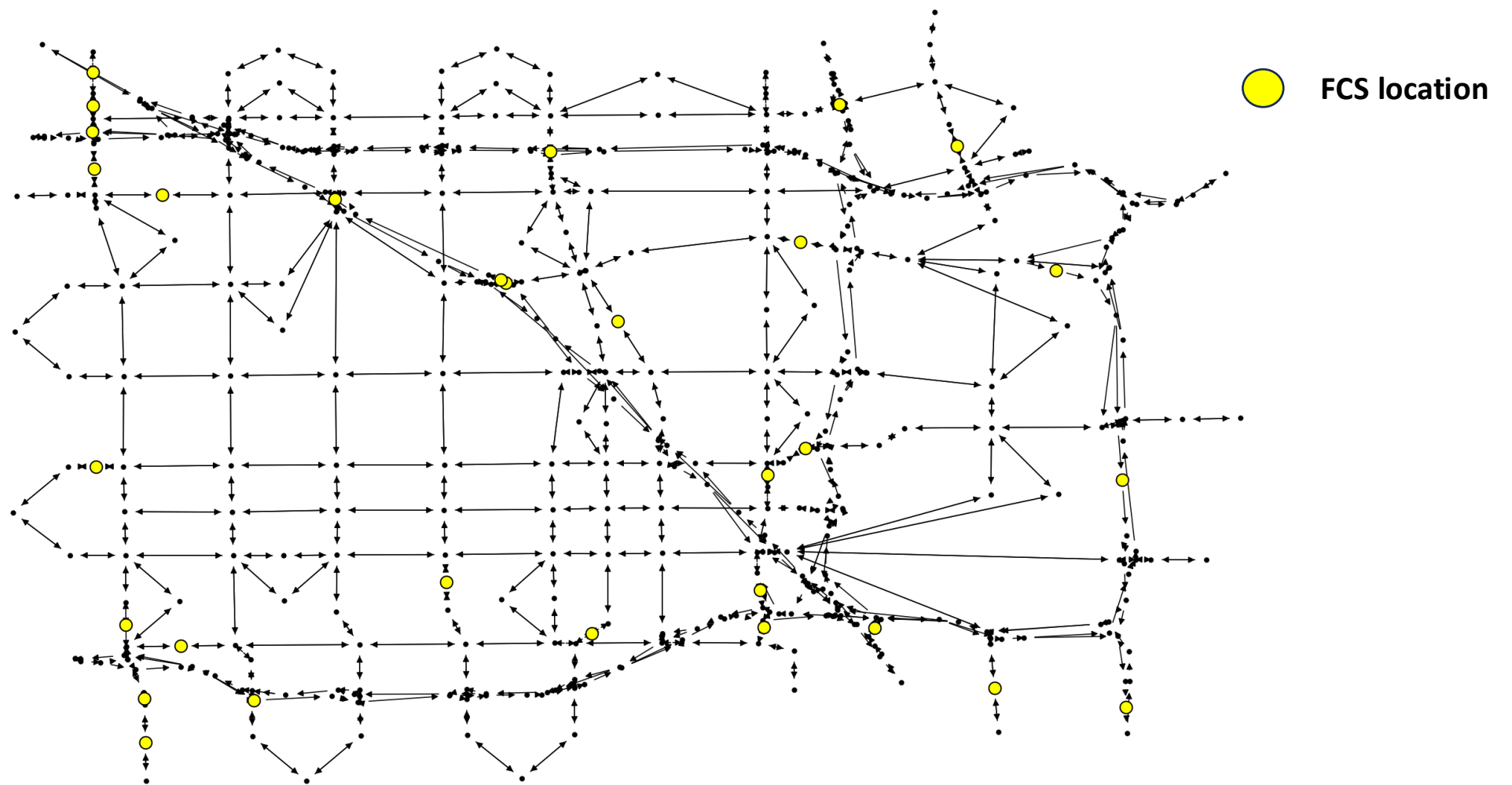
Thought for 21s

You're right—this was almost certainly a **node ID type mismatch** (e.g., 16 vs "16" or Excel's 16.0). I've patched the script to **normalize all node IDs to strings** everywhere (edges + FCS file) and added **verbose diagnostics** so you can see exactly which FCS rows matched and where each FCS was placed.

Download the fixed script:

[build_graph_with_fcs.py](#)

Share



Anaheim network with locations of charging stations

-  New chat
-  Search chats
-  Library
-  Codex
-  Sora
-  GPTs
-  image generator

 New project

Chats

 Joseph Moyalán
Plus

Plot the shortest route to safe shelter with given SoC of EV



I need to route an EV from a start node to a destination node. The EV has an initial driving range (distance). A path's length is the sum of its edge lengths and must not exceed the EV's available range. If the EV can't reach the destination, it may detour to the nearest fixed charging station (FCS). Each FCS visit adds 792,000 distance units to the range and incurs 0.5 hours of time.

Please provide code that takes the start node ID, destination node ID, and initial range, and returns the minimum-time path. Total time equals the sum of each edge's `free_flow_time` × 0.0166667, plus 0.5 hours for each FCS visit. Also, visualize the selected path in Networkx with its edges highlighted in blue.

Thought for 45s

Absolutely—here's a self-contained NetworkX routine that finds the **minimum-time** route with **range constraints** and optional **mid-path charging at FCS nodes**. It:

python  Copy code

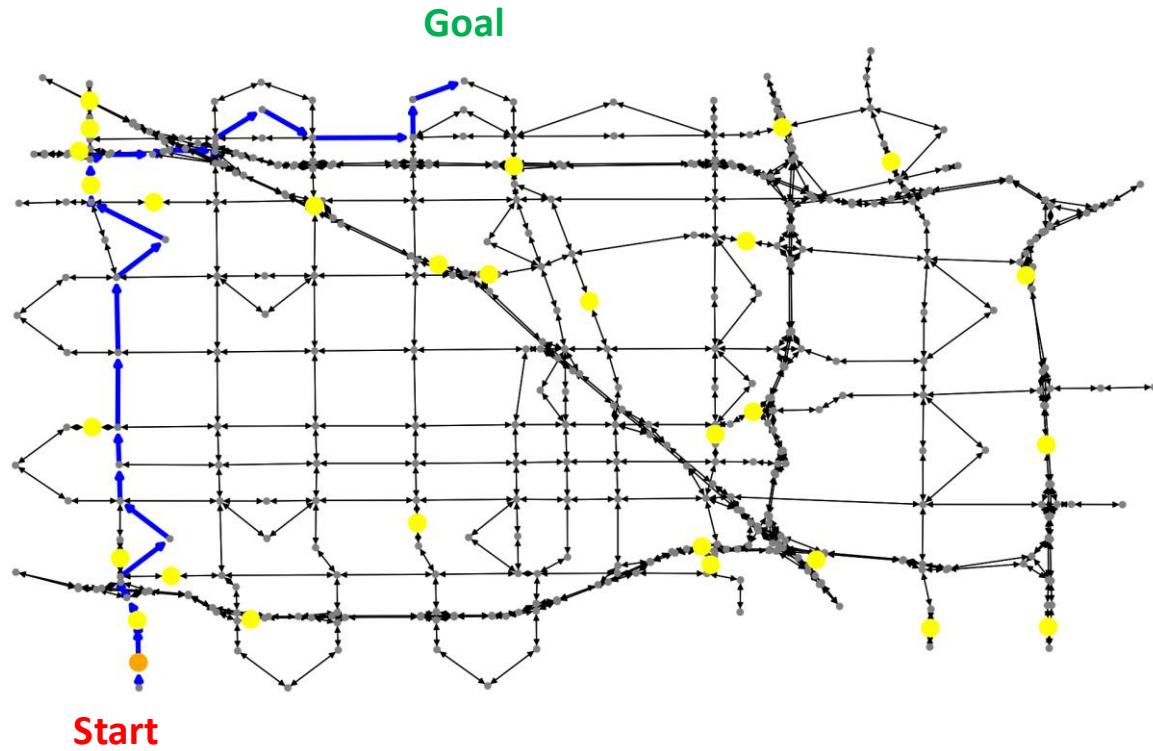
```
import heapq
import networkx as nx
import matplotlib.pyplot as plt
import math

FF_TO_HOURS = 0.0166667 # multiply edge free_flow_time by this to get hours
CHARGE_ADD = 792000.0 # distance units added per FCS visit
CHARGE_TIME = 0.5 # hours added per FCS visit

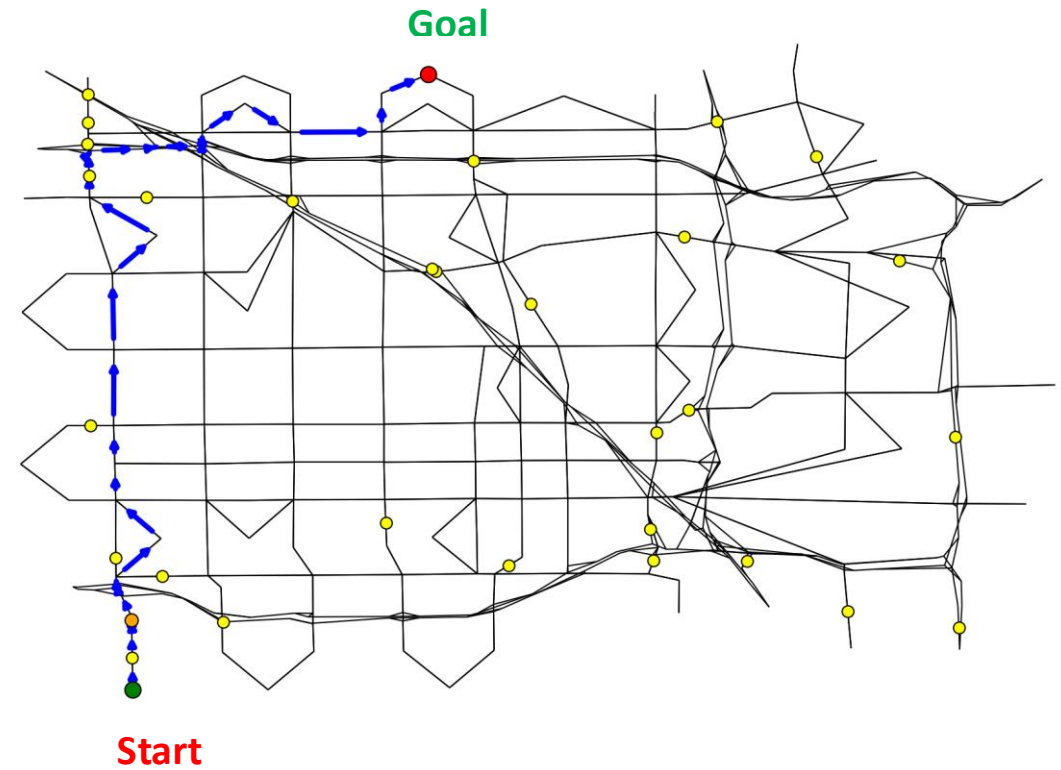
def _is_fcs_node(G, n):
    return G.nodes[n].get("type") == "FCS"

def min_time_path_with_fcs(
    G: nx.DiGraph,
    start_id
```

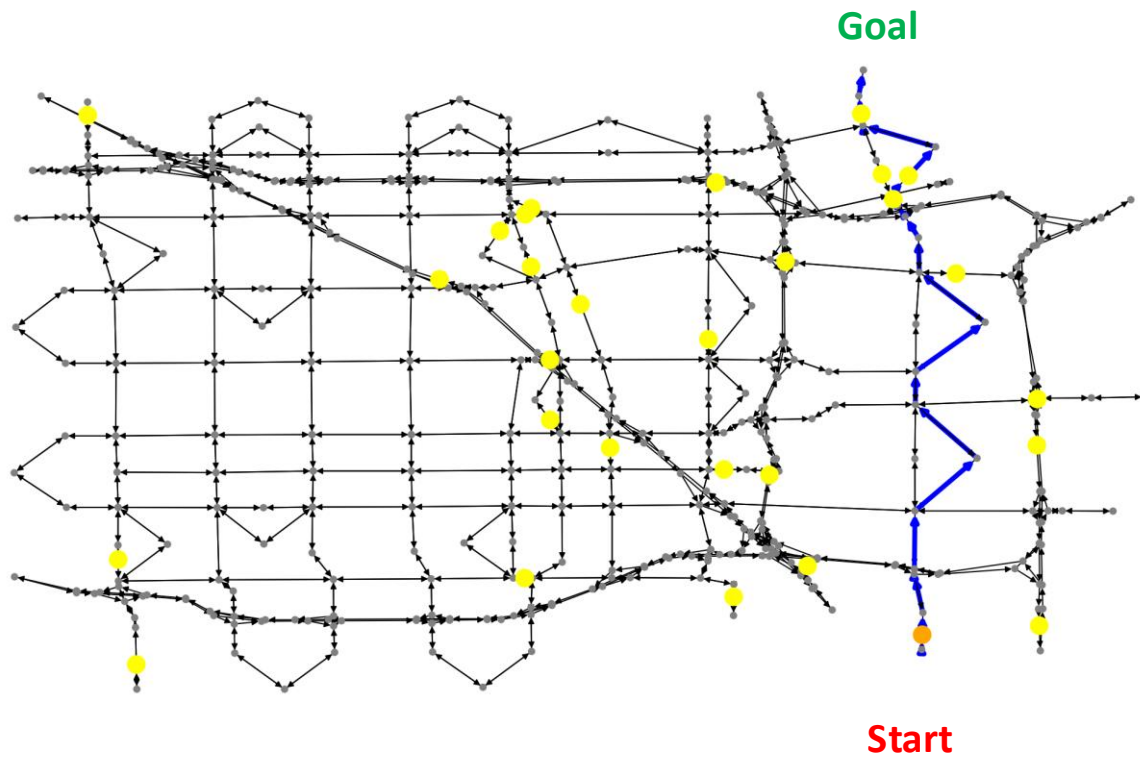
Our optimization-based approach



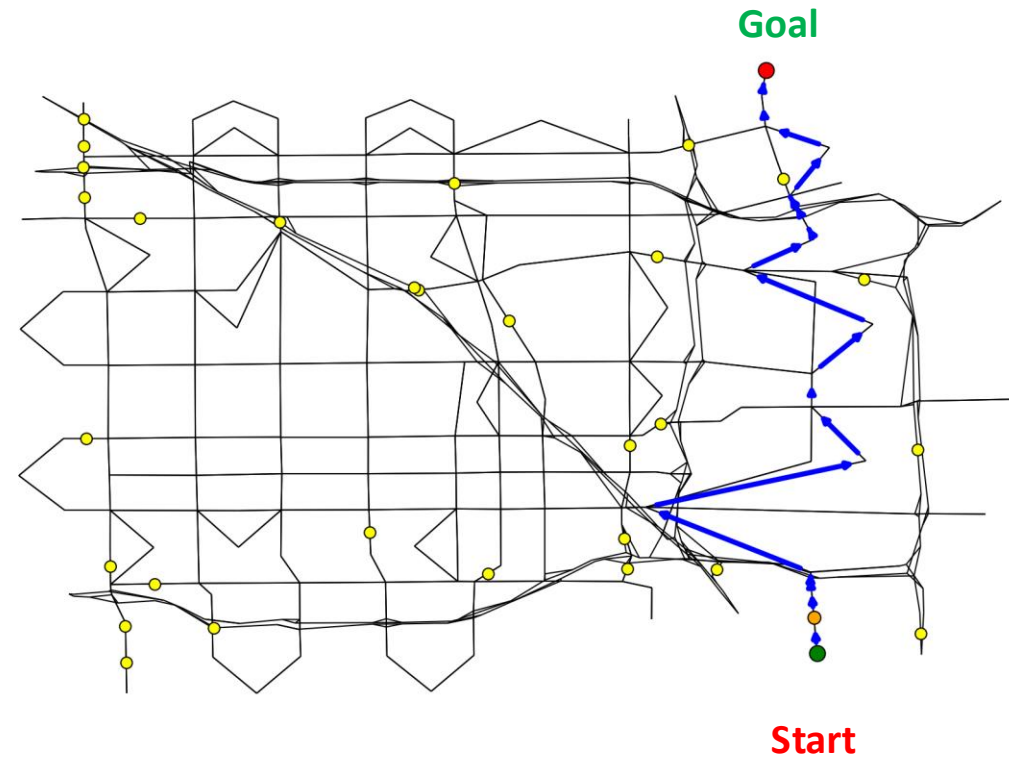
ChatGPT code



Our optimization-based approach

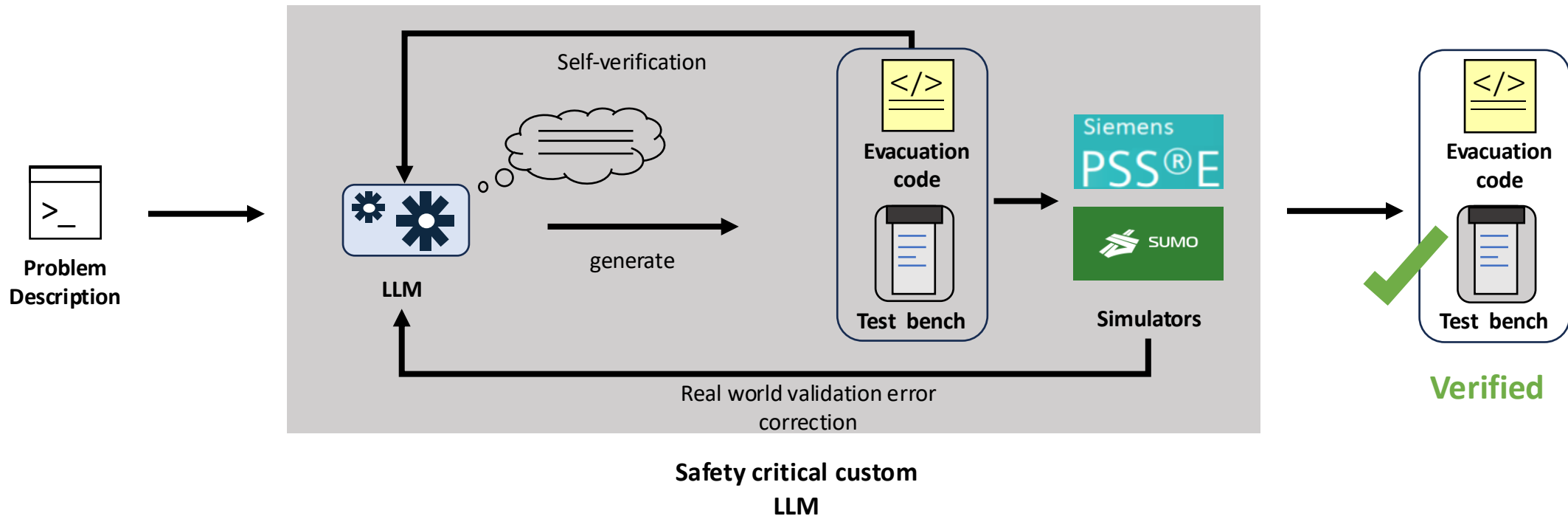
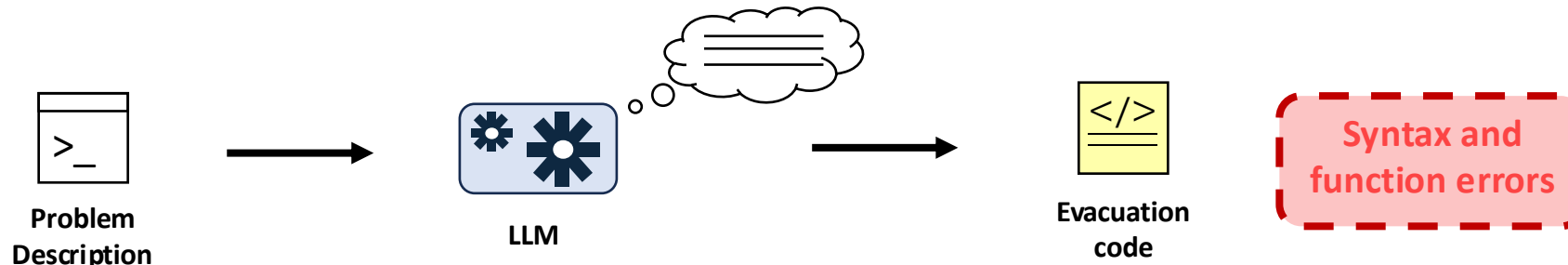


ChatGPT code



Custom made LLMs for deployment of EV chargers

Work in Progress



Conclusion

- General LLMs like ChatGPT are powerful but need detailed information for niche domains.
- Interpreting their solutions may require technical background or relevant skills.
- If input information is incorrect or vague, the solutions can be inaccurate and unreliable.
- Custom LLMs (e.g., for emergency evacuation) don't depend heavily on context.
- They only require problem-specific parameter specifications to provide solutions.

Thank you

jmoyalan@ucmerced.edu
rpintodecastro@ucmerced.edu